EX08 - Movimiento vertical - while y Euler

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex08_vertical_motion.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Simular un lanzamiento vertical integrando dh/dt=v y dv/dt=-g con Euler hasta que la altura sea negativa.

Variables / elementos del problema

Nombre	Significado
h	altura [m]
v	velocidad vertical [m/s]
t	tiempo [s]
g	gravedad [m/s^2]
dt	paso temporal [s]

Ecuacion o logica en papel

```
dh/dt = v
dv/dt = -g
h_new = h + v*dt
v_new = v - g*dt
t_new = t + dt
```

Comprobacion manual

Primer paso: h=0,v=50,t=0,dt=0.1 -> h_new=5.0, v_new=49.019, t_new=0.1.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- variables
- if/elif
- bucles

Nuevo en este ejercicio:

- while
- estado de simulacion
- Euler explicito
- contador i
- actualizar estado
- maximo
- error relativo

3. Conceptos Python nuevos

1. while

Sintaxis: `while h >= 0:`

Repite mientras la condicion sea True. Es apropiado cuando no sabemos de antemano cuantas iteraciones necesitaremos.

2. Estado

Sintaxis: `h, v, t`

El estado contiene la informacion necesaria para calcular el siguiente instante.

3. Euler

Sintaxis: `y_new = y + derivative * dt`

Euler aproxima la evolucion durante dt usando la derivada en el estado actual.

4. Contador

Sintaxis: `i += 1`

i cuenta iteraciones; no es una magnitud fisica.

4. Codigo paso a paso

Paso 1. Inicializar estado

```
t = t0
h = h0
v = v0
max_h = h0
i = 0
```

Antes del while necesitamos un estado inicial y variables auxiliares.

Paso 2. Condicion de parada

```
while h >= 0:
```

Mientras el estado actual este sobre el suelo se calcula otro paso.

Paso 3. Calcular estado nuevo

```
h_new = h + v * dt
v_new = v - g * dt
t_new = t + dt
```

Primero calculamos todas las variables nuevas usando el estado antiguo.

Paso 4. Actualizar maximo

```
if h_new > max_h:
    max_h = h_new
```

Solo se reemplaza max_h cuando aparece una altura mayor.

Paso 5. Avanzar estado

```
h = h_new
v = v_new
t = t_new
i += 1
```

El estado k+1 se convierte en el estado actual de la siguiente iteracion.

Paso 6. Validar con solucion analitica

```
analytical_h_max = h0 + v0**2 / (2 * g)
analytical_t_f = ...
analytical_v_f = ...
```

La solucion analitica es independiente del algoritmo Euler y permite medir su error.

5. Flujo de ejecucion

```
initial state
 -> while h>=0
 -> Euler step
 -> update max
 -> replace state
 -> repeat
 -> analytical comparison
```

6. Resultado / comprobacion esperada

Con dt=0.1, el archivo da max_h~129.92 m, t_f~10.30 s y v_f~-51.04 m/s tras 103 iteraciones.

7. Errores tipicos al empezar

- Actualizar h antes de calcular todas las variables nuevas.
- Olvidar incrementar t.
- No tener condicion de parada.
- Confundir error numerico con error de ejecucion.

8. Si solo recuerdas cinco cosas

- while se usa cuando la parada depende del estado.
- Euler transforma derivadas en pasos.
- El nuevo estado se calcula desde el actual.
- dt afecta al error.
- Siempre que puedas, valida con una solucion conocida.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.